

FitSense

Personalized fitness recommendations powered by AI

Abstract

FitSense (FitTrack AI) is a RESTful backend that replaces manual workout record keeping with a secure, centralized system and adds AI-generated recommendations and insights through Google Gemini. This report describes the problem addressed, the MVC-based architecture, the data model, the setup procedure and the API testing carried out.

1. Description

FitTrack AI is a secure, scalable, AI-powered RESTful backend for workout tracking and intelligent fitness recommendations. It is built with **Node.js**, **Express.js**, **MongoDB** and **Mongoose**, follows the **MVC architecture**, and exposes APIs for authentication, workout management, workout search and AI-assisted guidance.

Authentication uses **JWT**, passwords are hashed with **bcrypt.js**, and every protected API is validated by middleware. Integration with **Google Gemini AI** gives personalized workout plans from a user's age, goal and experience level, and fitness insights from statistics such as total workouts, average duration and calories burned. Responses are standardized JSON, so the backend works with web, mobile and third-party clients.

2. Scenario-Based Case Study

Background: Rahul exercises regularly but records workouts in notebooks and spreadsheets. He struggles to organize his history, monitor progress, check whether his habits match his goals, and choose workouts for his fitness level without professional guidance.

Problems	Solution
No centralized workout record; manual tracking is inaccurate and slow (calories, duration); no secure login for personal data; past workouts are hard to search; no personalized recommendations or performance insights.	A central backend where users register, log in and manage workouts (add, update, delete, retrieve, search) through REST APIs. Gemini AI analyzes user data and statistics to give recommendations and insights, secured by JWT, password hashing, middleware authorization, centralized error handling and MongoDB schema validation.

Usage: After registering and logging in, the user receives a JWT that authorizes all protected APIs: workout CRUD, search by name/category/date, **AI recommendations** (input: age, goal, experience; output: workout plan, weekly suggestions, level-based advice, motivation, safety tips) and **AI insights** (input: total workouts, average duration, calories; output: personalized performance analysis).

Outcome: Manual record keeping is eliminated, personal data stays private, and users get AI-guided routines. The modular design can later add nutrition tracking, BMI, wearable integration, reminders and analytics dashboards.

3. System Requirements

Type	Requirement
Software	Windows 10/11, macOS or Linux; Node.js v16+; npm v8+; Express.js ; MongoDB (users and workouts); Postman (API testing); VS Code or similar editor.
Hardware	Intel Core i5 (8th Gen+) / AMD Ryzen 5 or equivalent; 8 GB RAM minimum (16 GB recommended); 1 GB free storage.

4. Project Architecture

Requests reach the Express server, pass through the JWT authentication middleware on protected routes, and go to controllers. Controllers call Mongoose models or the Gemini service layer and return a structured JSON response. Each layer has a single responsibility, which keeps coupling low.

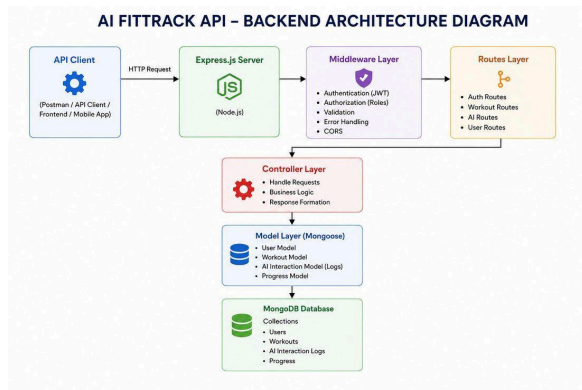


Figure 1: Backend architecture

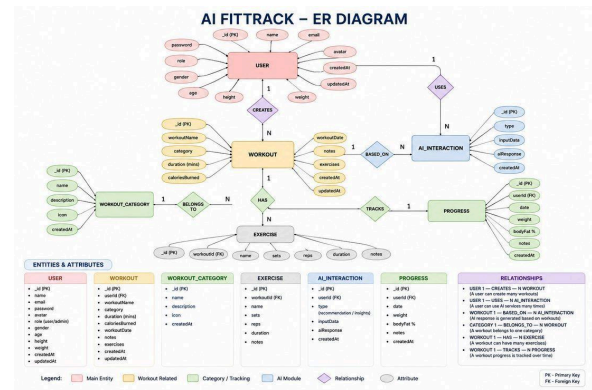


Figure 2: ER diagram

Component	Responsibility
Client Layer	React/mobile app or Postman; sends requests, stores the JWT, displays JSON responses and calls AI APIs.
Express Server	Starts the server, parses JSON, enables CORS, registers routes and middleware, handles errors centrally.
Auth Middleware	Reads the Authorization header, verifies the JWT, attaches user info to the request, rejects unauthorized calls.
Routes	Auth (register, login, profile); Workout (add, get all, search, get by ID, update, delete); AI (recommendation, insights).
Controllers	Auth, Workout and AI controllers: validate input, run business logic, call services/models, return JSON.
Services	Gemini AI service (prompts and responses), JWT service (create/verify tokens), Password service (hash/compare).
Database	MongoDB with Mongoose ODM; Users and Workouts collections; flexible schema, scalable.

5. Data Model

Two collections are linked by a **one-to-many** relationship: one **User** can own many **Workouts** (e.g. running, cycling, strength), and each workout stores its owner's ObjectId in the **user** field.

User entity

Field	Type	Description
_id	ObjectId	Primary key
name	String	Required
email	String	Unique
password	String	bcrypt hash
createdAt / updatedAt	Date	Timestamps

Workout entity

Field	Type	Description
_id	ObjectId	Primary key
user	ObjectId	Ref: User
workoutName	String	Required
category	String	Required
duration	Number	Minutes, > 0
caloriesBurned	Number	>= 0
workoutDate	Date	Required

6. Key Features, Roles and MVC

Feature	Details
Secure authentication	Registration, login, password encryption, JWT generation, protected APIs.
Workout management	Full CRUD, each workout linked to its authenticated user.
Workout search	By name, category or date.
Gemini recommendations	Input: age, goal, experience. Output: weekly plan, exercises, training and safety tips.
AI fitness insights	Input: total workouts, average duration, calories. Output: analysis, improvements, progress summary.
Error handling	JSON errors for invalid credentials, missing token, validation, database and server errors.

Feature	Details
REST and modular design	GET, POST, PUT, DELETE with JSON; separate models, controllers, routes, middleware, services, config.

Role: The single role, **Authenticated User**, can register, log in, view the profile, add/view/search/update/delete workouts, and request AI recommendations and insights. Every operation requires a valid JWT.

MVC: The **Model** (Mongoose User and Workout schemas) handles data and validation; the **View** is the API interface consumed by React, mobile apps or Postman; the **Controller** validates requests, invokes services and models, and returns JSON.

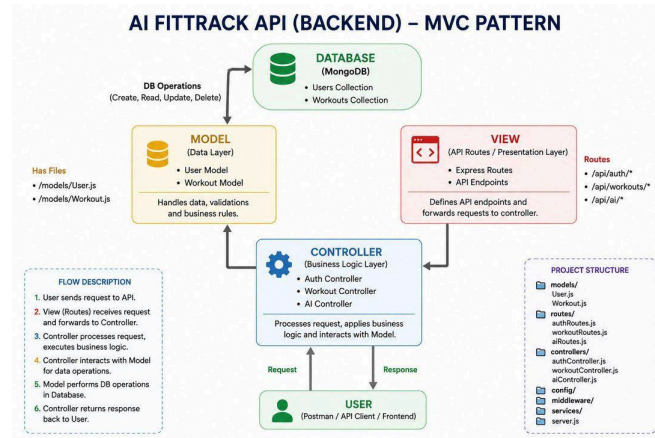


Figure 3: MVC pattern

7. Project Setup and Database

- Setup:** create the folder **AI-FitTrack-API**, open it in VS Code, create a **server** folder, and run these in it:

```
npm init -y
npm install express mongoose dotenv bcryptjs jsonwebtoken cors
npm install --save-dev nodemon
```

- .env** (keeps secrets out of source code):

```
PORT=5000 MONGO_URI=mongodb://localhost:27017/aifittrack
JWT_SECRET=your_secret_key GEMINI_API_KEY=your_google_gemini_api_key
```

- Database:** MongoDB stores JSON-like BSON documents; Mongoose adds schemas, validation and CRUD helpers. The connection lives in **config/db.js** (connect, handle errors, show status).
- User schema** (models/User.js): name required, email unique, password required and hashed with bcrypt.js, timestamps on.
- Workout schema** (models/Workout.js): name, category and date required, duration > 0, calories >= 0, each workout belongs to one user.

8. Execution and API Testing

Install Postman or ThunderClient, start the server with **npm start**, then test in order: Authentication APIs, Workout APIs, AI APIs.

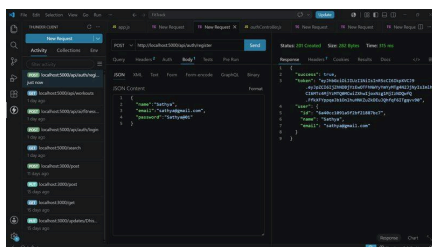


Figure 4: Register

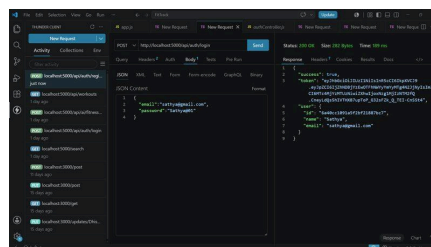


Figure 5: Login

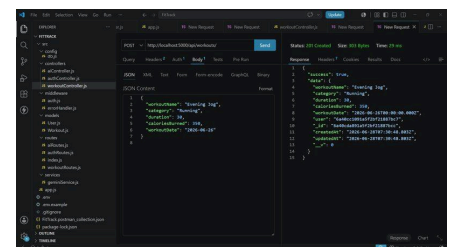


Figure 6: Add workout

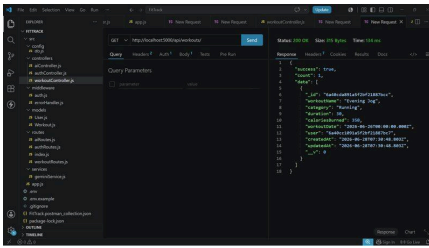


Figure 7: Get all workouts

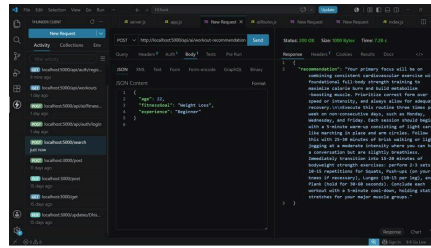


Figure 8: AI recommendation

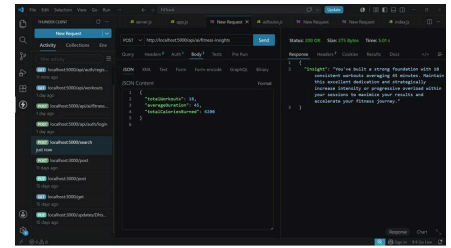


Figure 9: AI fitness insights

9. Conclusion

FitTrack AI removes manual tracking, protects personal workout data with JWT authentication and hashed passwords, and gives users Gemini-powered plans and insights. All authentication, workout and AI endpoints were verified in Postman. The modular MVC design allows later extensions such as nutrition tracking, BMI calculation, wearable integration, reminders and analytics dashboards.

References and Links

Demo video:

https://drive.google.com/file/d/1EoynUI0RtPdce0rUSx9CLH3AHc5GPsVw/view?usp=drive_link

Code:

https://drive.google.com/drive/folders/1NU6h78XO4HwP_GIGv1kgX_hNacTbj2uc?usp=drive_link